When you have EF Core configured and a database created, section 12.4 shows how to use it in your application code. You'll see how to *create, read, update, and delete* (CRUD) records, and you'll learn about some of the patterns to use when designing your data access.

In section 12.5 I highlight a few of the problems you'll want to take into consideration when using EF Core in a production app. A single chapter on EF Core can offer only a brief introduction to all the related concepts, so if you choose to use EF Core in your own applications—especially if you're using such a data access library for the first time—I strongly recommend reading more after you have the basics from this chapter.

Before we get into any code, let's look at what EF Core is, what problems it solves, and when you may want to use it.

12.1 Introducing Entity Framework Core

Database access code is ubiquitous across web applications. Whether you're building an e-commerce app, a blog, or the Next Big Thing™, chances are that you'll need to interact with a database.

Unfortunately, interacting with databases from app code is often a messy affair, and you can take many approaches. A task as simple as reading data from a database, for example, requires handling network connections, writing SQL statements, and handling variable result data. The .NET ecosystem has a whole array of libraries you can use for this task, ranging from the low-level ADO.NET libraries to higher-level abstractions such as EF Core.

In this section, I describe what EF Core is and the problem it's designed to solve. I cover the motivation for using an abstraction such as EF Core and